Fused AllReduce + Residual + RMSNorm

One NVLink-native op for the tensor-parallel layer boundary

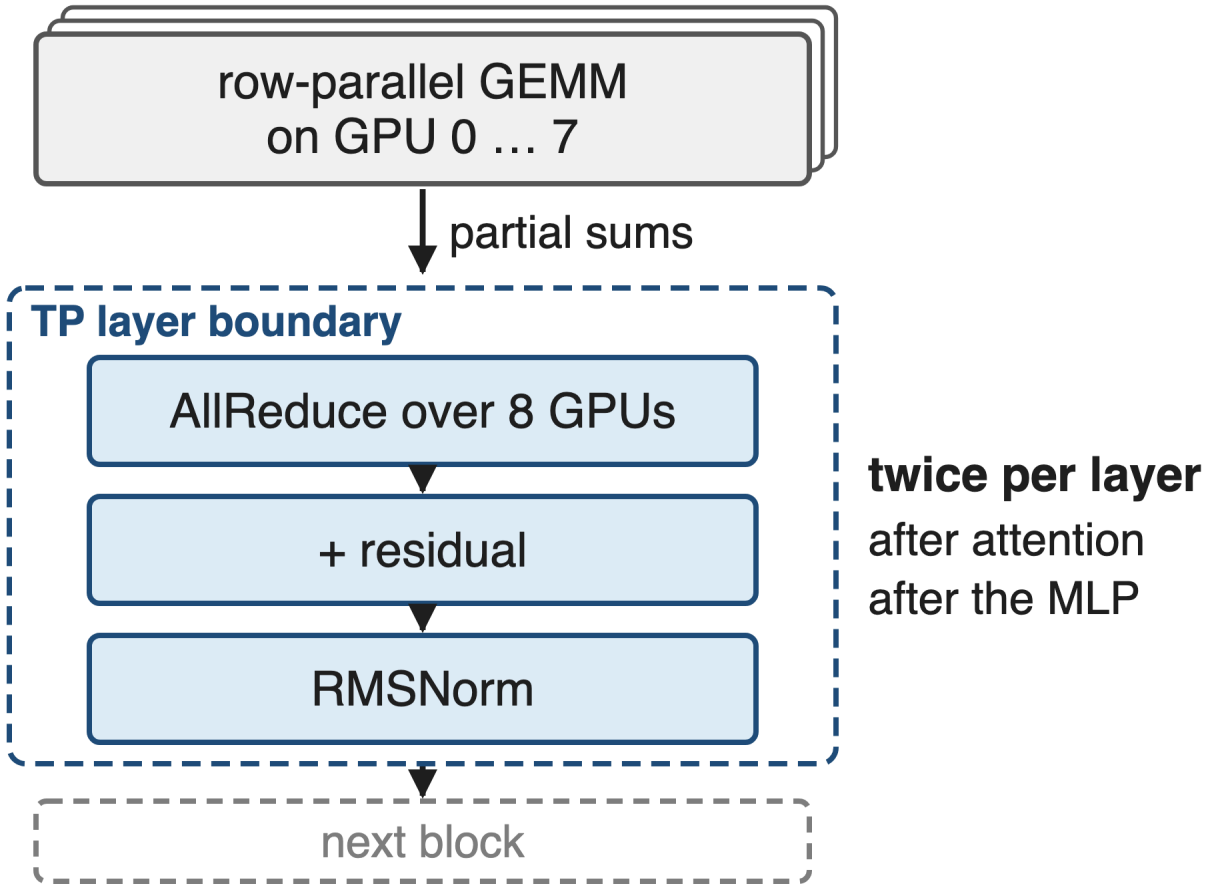
Tencent Hunyuan AI Infra · HPC-Ops

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Comm/Compute Overlap · github.com/Tencent/hpc-ops

Every TP block ends in AllReduce, add, RMSNorm

- Target: TP over one **8×H20** NVLink node
- **2** AllReduces per layer, each followed by add + RMSNorm
- BF16, hidden **4096 / 5120 / 7168**



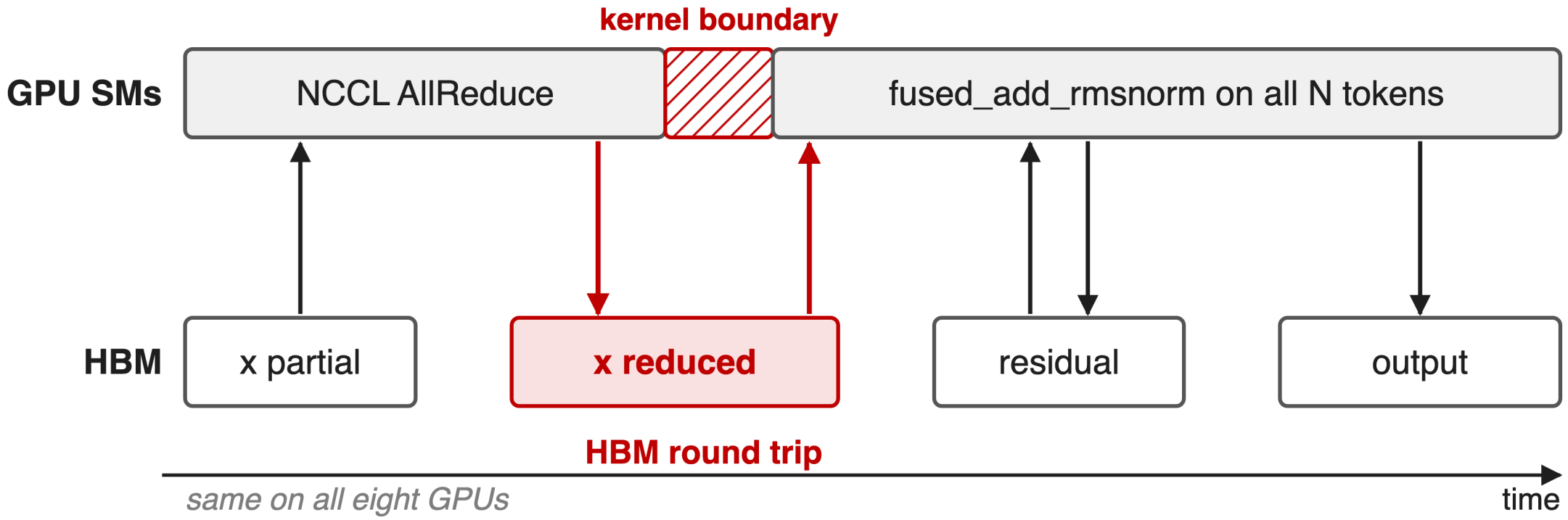
This boundary is worth 5–20% end-to-end

5–20%

end-to-end speedup vLLM lists for fusing just AllReduce + RMSNorm, at small token counts

vLLM docs v0.19.1, "Fusion torch.compile passes", quick-reference table (indicative, model- and hardware-dependent)

Unfused, the sum round-trips through HBM



Two kernels, one HBM round trip, and every GPU normalizes all N tokens

Prior fused kernels stop at decode sizes

- **NCCL + separate norm**
- Any token count
- **2 kernels, HBM round trip**

- **FlashInfer / TRT-LLM fused**
- vLLM fuses **only small N**
- mnnvl: ≤ **8,192** tokens on H20

8 tokens = 112 KiB, 32K = 448 MiB: no single schedule fits both

Insight: two-shot hands each GPU whole tokens

- RMSNorm needs one token's full hidden vector, nothing more
- Shot 1 (reduce-scatter) leaves each GPU complete sums of its own tokens

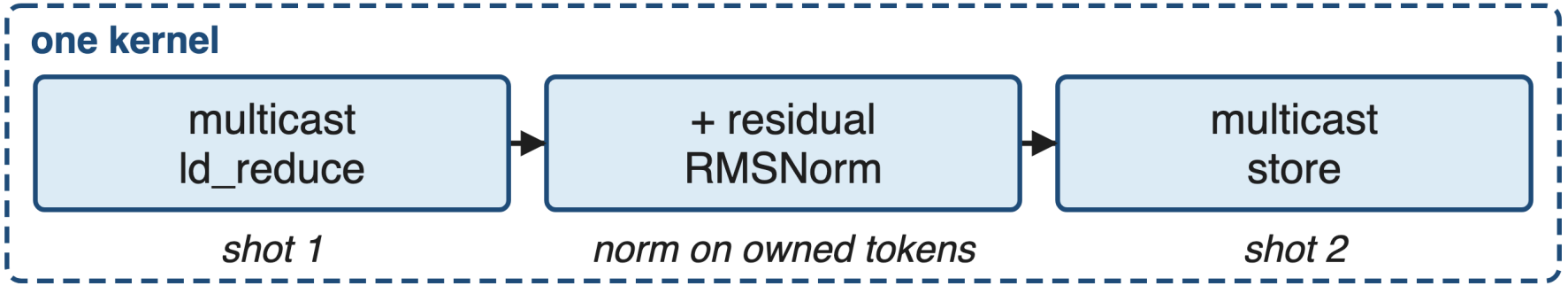
Insight: two-shot hands each GPU whole tokens

- RMSNorm needs one token's full hidden vector, nothing more
- Shot 1 (reduce-scatter) leaves each GPU complete sums of its own tokens

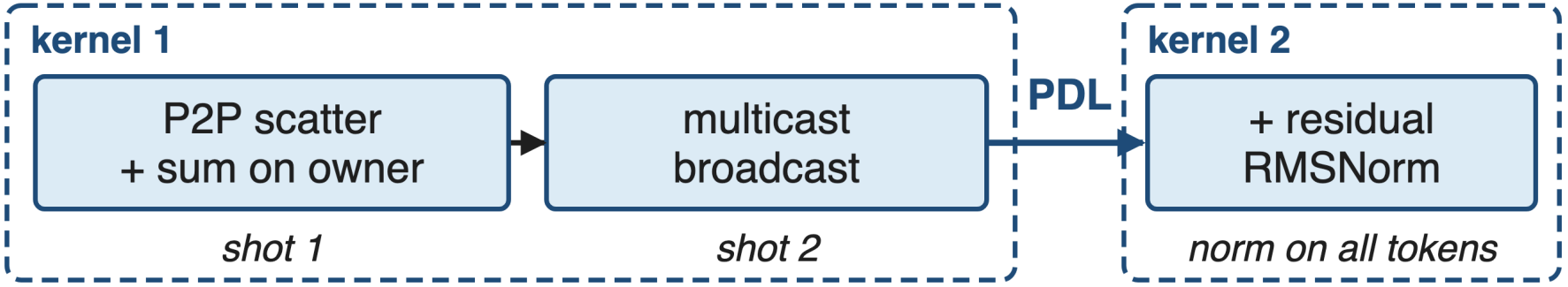
Normalize inside the collective, and pick the transport by token count

One op, two transports, one two-shot schedule

2 high-throughput
many tokens

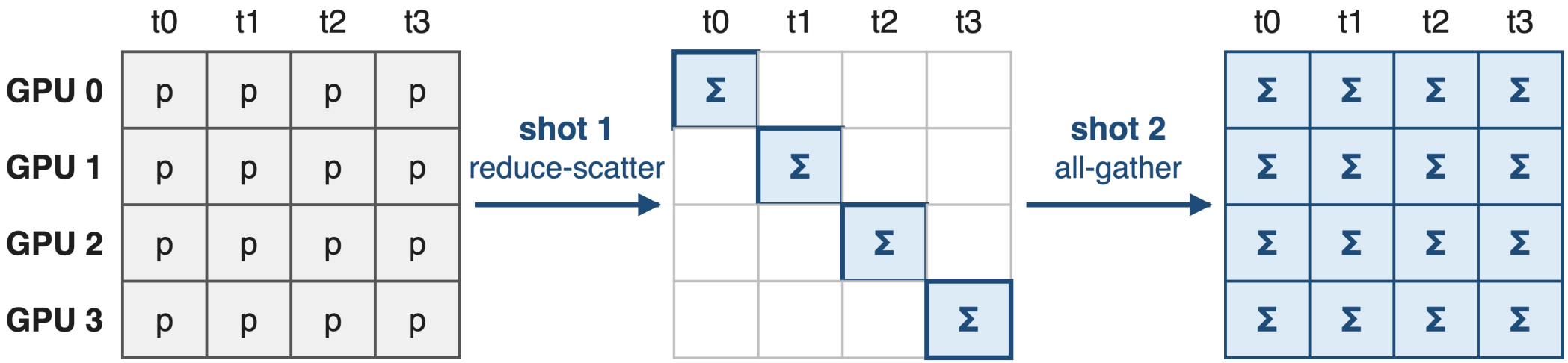


3 low-latency
few tokens



HT = fuse_allreduce_rmsnorm_high_throughput · LL = fuse_allreduce_rmsnorm_low_latency · BF16, one NVLink node

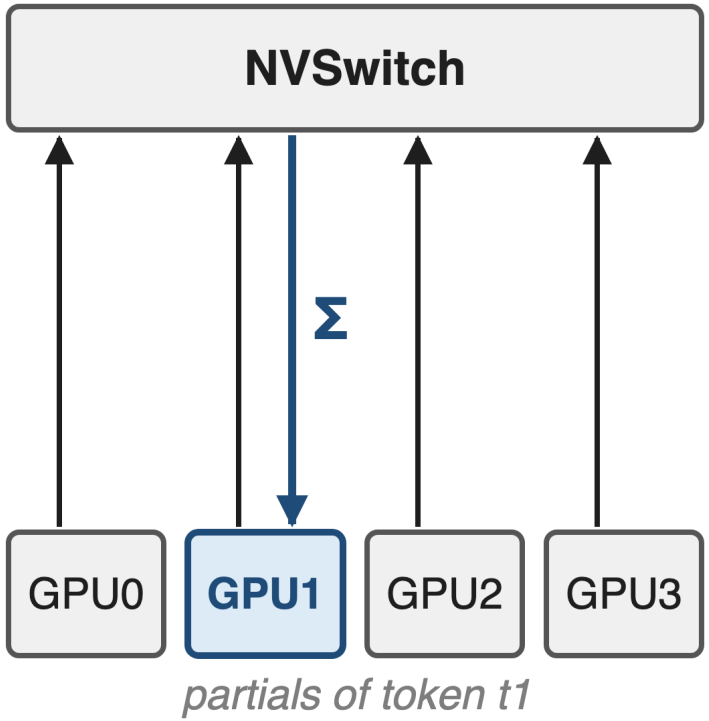
❶ Two shots give every token one owner



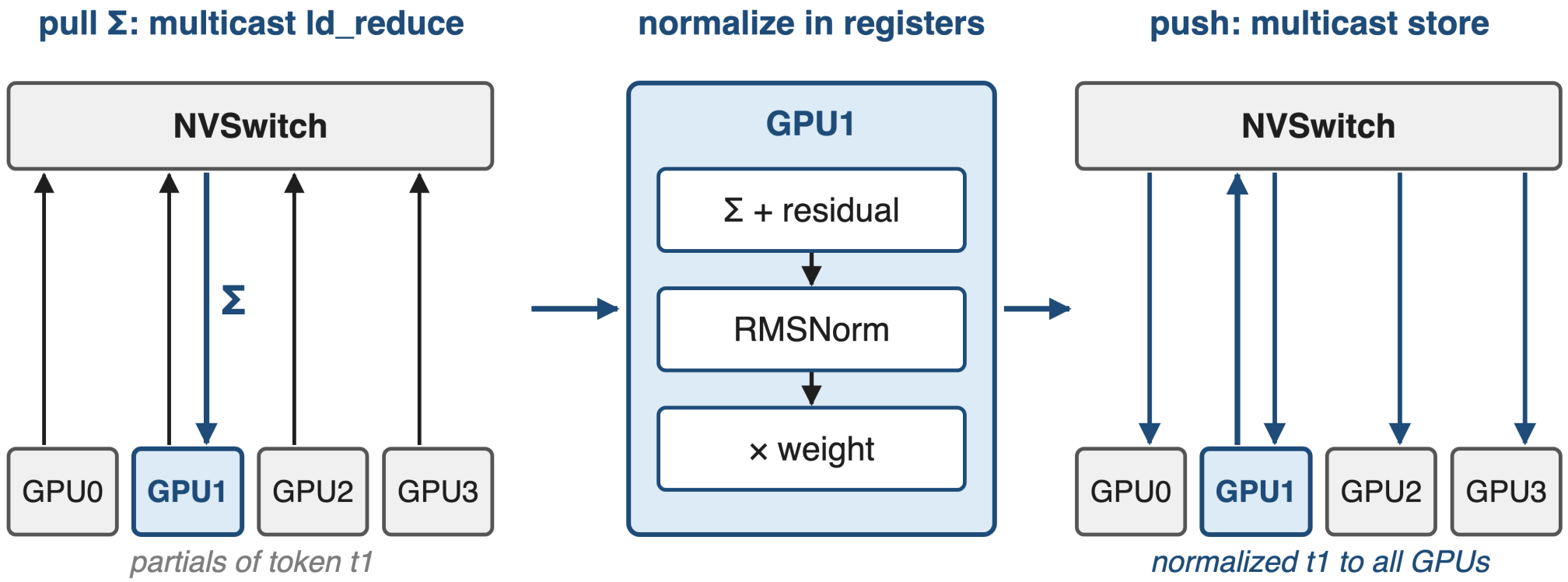
HT: the owner adds the residual and normalizes between the two shots

❷ HT: the switch sums, the owner normalizes

pull Σ: multicast ld_reduce

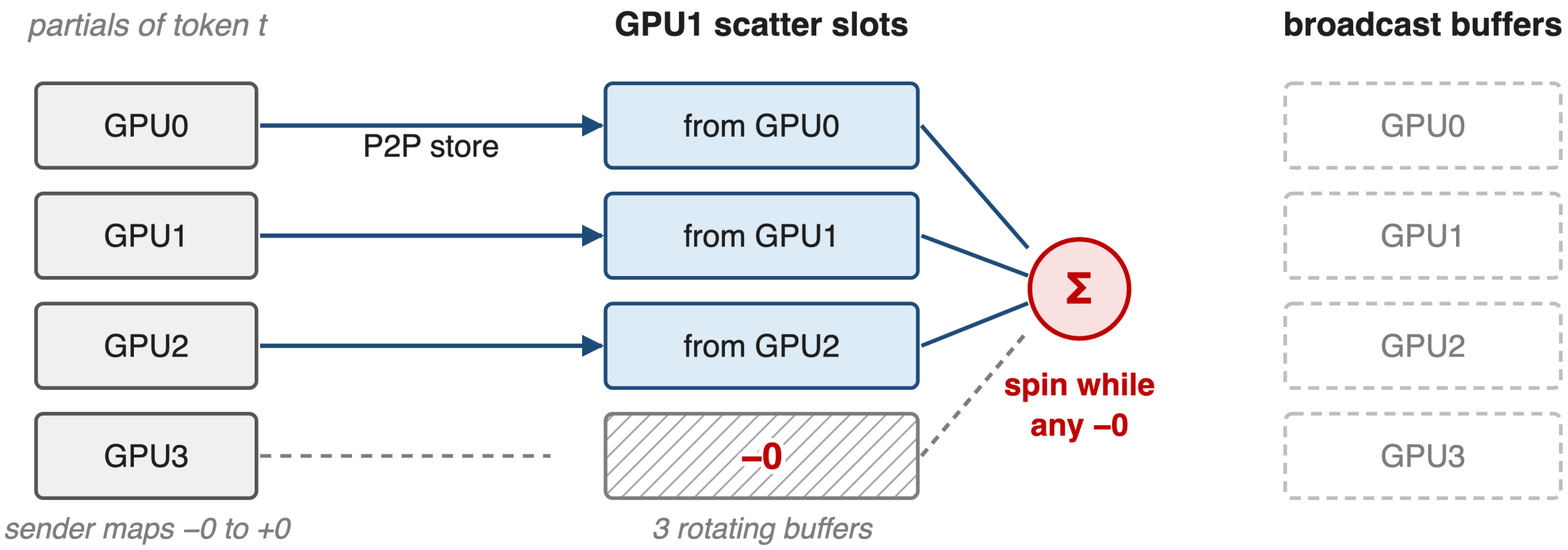


❷ HT: the switch sums, the owner normalizes

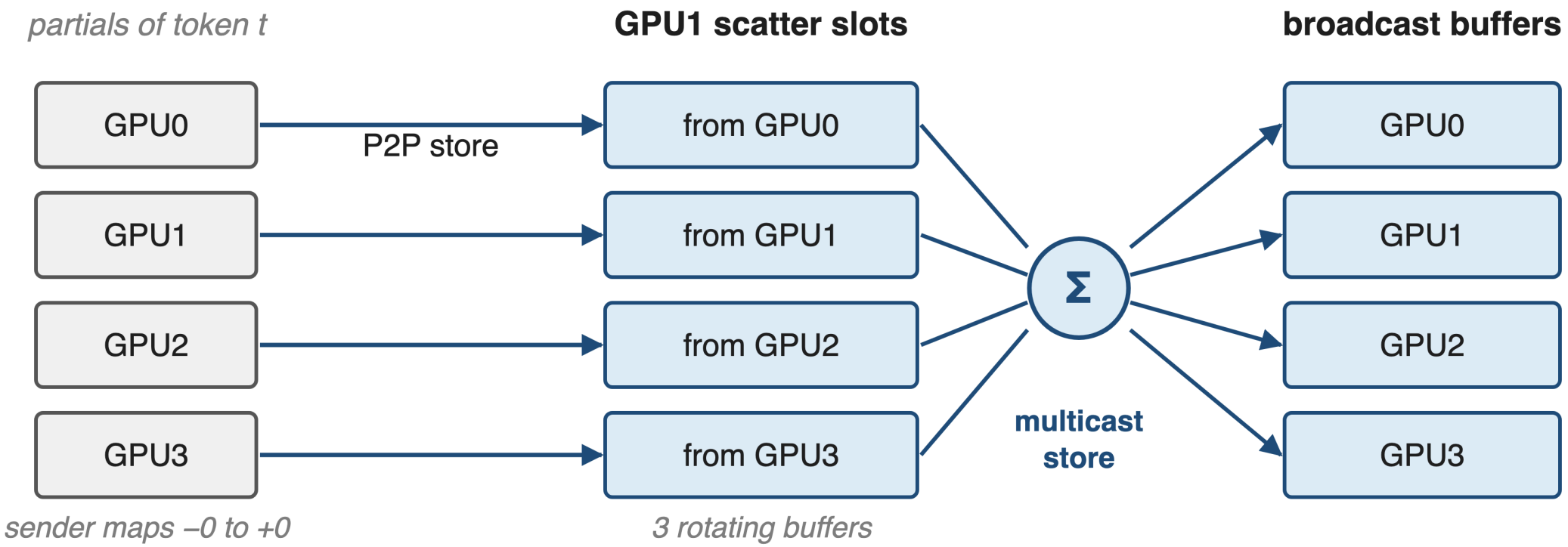


One kernel: the reduced tensor never lands in HBM

❸ LL: the data is its own ready flag

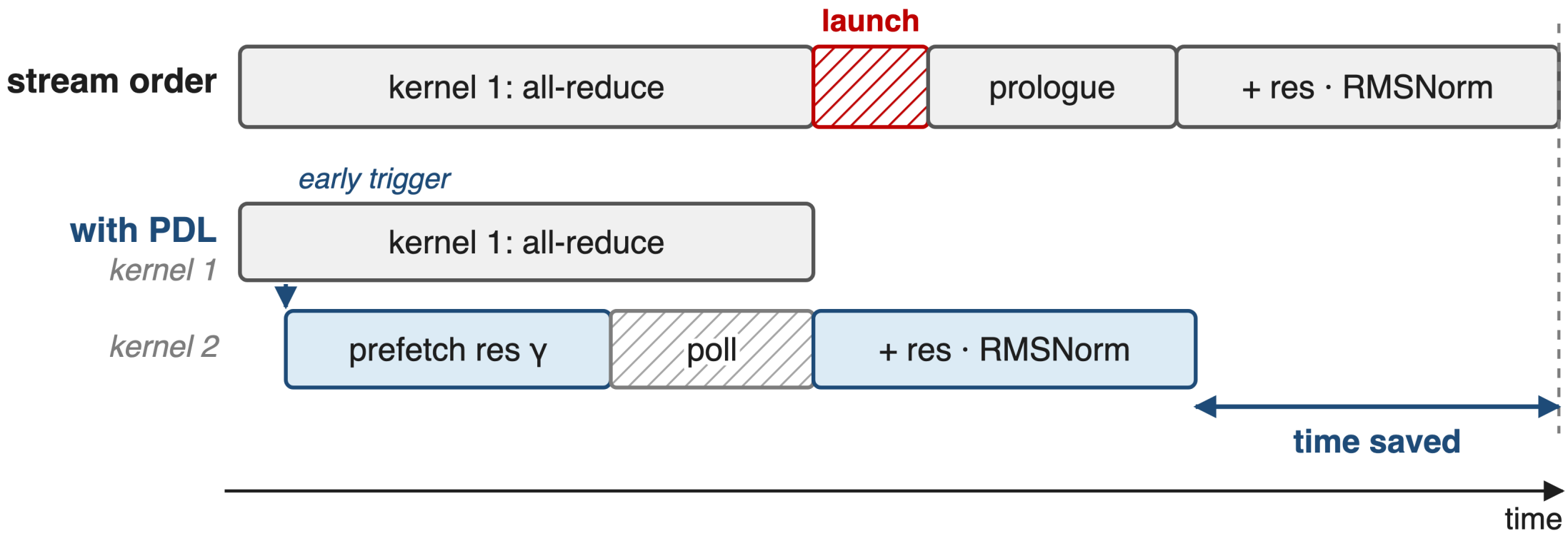


❸ LL: the data is its own ready flag



No separate flag write and no barrier before the sum

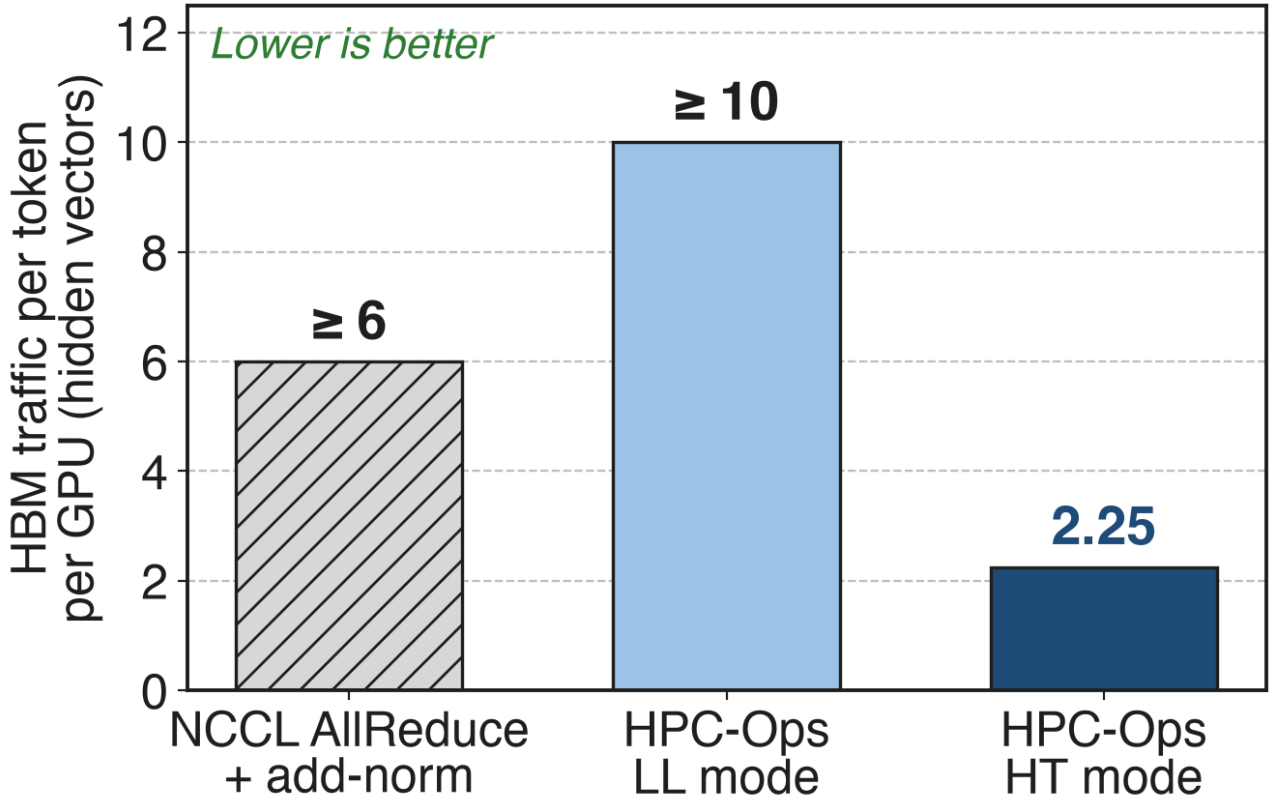
❸ PDL launches the norm kernel early



Schematic, not to scale. Kernel 2 also polls the Lamport broadcast buffer instead of waiting for kernel 1 to finish

HT cuts HBM traffic per token ≥ 2.7×

- Large N is bandwidth-bound: bytes decide
- LL spends bytes on sentinels to save sync
- So: **HT** for prefill, **LL** for decode



Analytic, from kernel code, TP = 8; unit = one BF16 hidden vector; NCCL internals and polling re-reads not counted

Up to 1.76× over NCCL and FlashInfer

1.76×

best HPC-Ops mode vs best of NCCL + norm and FlashInfer fused AllReduce, BF16 TP on one 8×H20 node

HPC-Ops README, Performance table (hidden 4096 / 5120 / 7168); benchmark/fuse_allreduce_rmsorm (hpc_best_speedup)

The price: one node, three shapes, two modes

- One NVSwitch node with multicast
- BF16; hidden **4096 / 5120 / 7168** only
- LL: power-of-two TP, **2–64**
- Inputs live in symmetric multicast buffers
- LL workspace: **6×** the activation
- Caller picks HT or LL; not yet in SGLang

Summary

- TP layer boundary: fusion worth **5–20%** end-to-end (vLLM)
- **Insight**: two-shot gives each GPU whole tokens, so normalize inside the collective
- **HT**: multicast reduce-norm-store · **LL**: Lamport P2P + PDL
- Up to **1.76×** over NCCL / FlashInfer on 8×H20

https://github.com/Tencent/hpc-ops